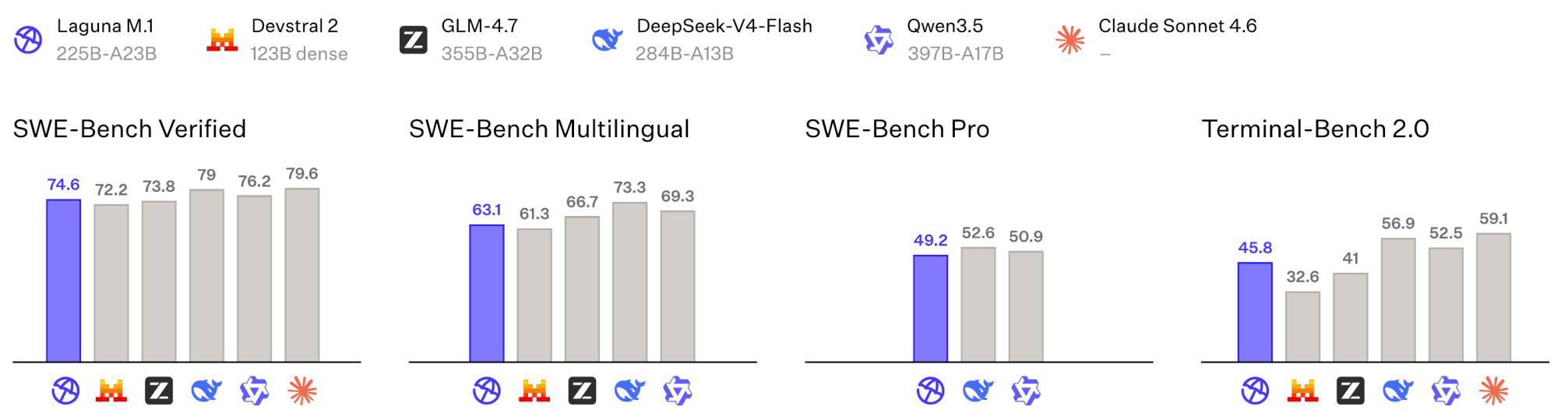
Laguna and Beam

arxiv.org/abs/2605.27605

Results: Laguna M.1

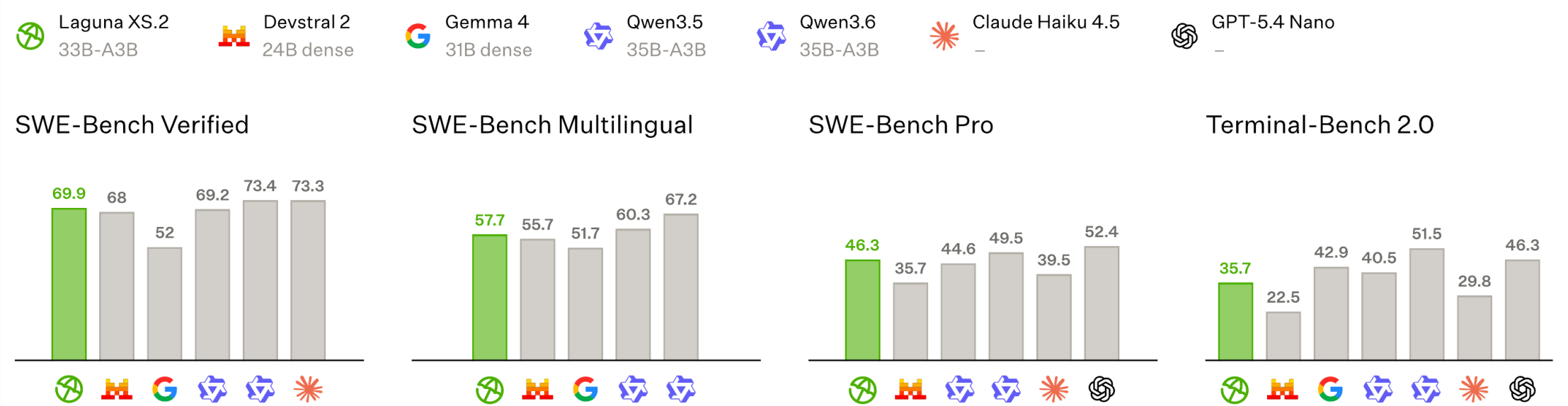


(a) M.1 results.

Also at ~20B active (self-reported)	SWE-V	Multilingual	Pro	Terminal-Bench
Laguna M.1 · 226B / 23B · Apr 2026	74.6	63.1	49.2	45.8 (2.0)
Beam · 501B / 23B · Oct 2026	81.6	78.3	66.1	80.1 (2.1)
Inkling 2 Small · 594B / 17B · Nov 2026	n/r	n/r	86.3 (v2-Hard)	n/r

Laguna Figure 1a

Results: Laguna XS.2



(b) XS.2 results.

beats Haiku 4.5 on the hard benchmarks, trails Qwen3.6.

Laguna Figure 1b

Outline

1. Model Factory · 5 min

Principles

The stack: Titan, Atlas, Blender, Spark, Hive, etc

2. Training details · 20 min

Architecture

Data: ranking, synthetic, AutoMixer

Infrastructure

Pre-training

Post-training

Quantization and evaluation

Features Beam

Model Factory: Three Principles

Treat model building as an industrial process.

1. Experiments as code	Every run is a committed config with an ID (use Dagster)
2. Composable components	Monorepo for research and production
3. Humans for novel decisions	Custom workload scheduler - sub-minute job placement

Custom Scheduler: Sub-minute Job Placement

	Volcano, tens of minutes	In-house, under a minute
Eviction	Whole nodes	Single jobs
Topology	etcd: latency tail under churn	FoundationDB: reads and writes without the tail
Pod death	Re-pack the whole gang	Respawn on the same node

Buys: hyperparameter sweeps, CI canaries, preemption-driven backfill.

Model Factory: The Stack

Layer	Component	What it does
Control plane	Dagster	Every run and artifact is a tracked asset
Scheduler	In-house	Sub-minute job placement
Data pipelines	Spark	All data processing; ~20T tokens a day
Synthetic data	Hive	Synthetic pipelines as config
Train and serve	Titan + Atlas	TorchTitan and vLLM forks sharing one model code; pre-training, SFT, RL, evals

Laguna §2, §3.1.2, §3.2, §4.4

Hive: Synthetic Data as Config

G generator (LLM call) · **V** verifier (judge or tests) · **f** filter (keep or drop) · **pre / post** setup and cleanup · ∘ read right to left

```
cascade P = post ∘ f_keep ∘ G_QA ∘ ( f[V ≥ τ] ∘ V ∘ G_textbook ) ∘ pre  # ~100B
    # paper → textbook section → judge, drop if < τ → Q&A → keep if answerable
```

```
rewrite P = post ∘ f_quality ∘ G_rewrite ∘ pre  # ~1T
    # web or STEM doc → rewrite in a new voice or structure → quality check
```

```
convert P = post ∘ f_format ∘ G_convert ∘ pre_seed  # ~10B
    # math problem, checked convertible → same problem as code → format check
```

```
loop    P = f_bounds ∘ loop_T( G_step ∘ G_orch ) ∘ G_init ∘ f_relevance  # ~10B
    # stack trace, relevance-checked → open a chat → orchestrator picks each turn, T rounds → bounds check
```

Laguna §3.2.2, Table 2, Figure 5

Outline

1. Model Factory

Principles

The stack: Titan, Atlas, Blender, Spark, Hive, etc

2. Training details

Architecture

Data: ranking, synthetic, AutoMixer

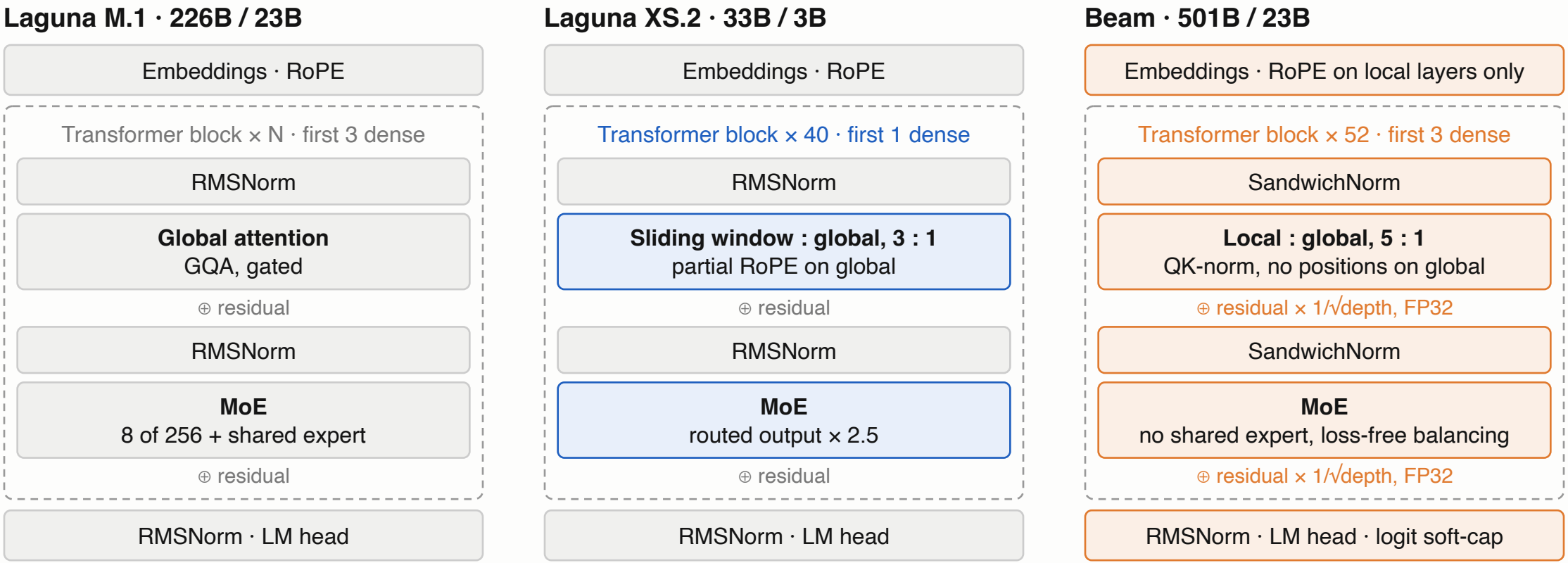
Infrastructure

Pre-training

Post-training

Quantization and evaluation

Architecture



■ Carried over ■ Changed in XS.2 ■ Changed in Beam

Laguna §3.1, Appendix A.2 · Beam §2.1.1, Table 1

RMSNorm vs SandwichNorm

```
def rms(x, g):   # RMSNorm: rescale to unit RMS, then a learned gain
    return g * x / sqrt(mean(x**2) + eps)   # no mean subtraction, no bias (vs LayerNorm)
```

```
# Laguna: pre-norm
# normalise only the input of each sub-layer f (attn or moe)

h_n = f(rms(x, g_in))
g_in = 1 at init,   no g_out

# the residual is a plain sum of every h
x_n = h_1 + h_2 + … + h_n   # grows with depth
```

```
# Beam: SandwichNorm + depth-scaled residual
# normalise the input AND the output of f
h_n = rms(f(rms(x, g_in)), g_out)
g_in = 1,  g_out = 1/√L at init

# the sum of every h, rescaled; kept in FP32
x_n = (h_1 + … + h_n) / √n   # RMS stays ~constant
```

Laguna §3.1 · Beam §2.1.1, Eq. 2.4

Attention

```
for l, block in enumerate(blocks):   # Laguna 3 : 1, Beam 5 : 1
    is_global = (l + 1) % (RATIO + 1) == 0
    h = x + block.attention(norm(x), window=None if is_global else 512)
    x = h + block.moe(norm(h))
```

```
def attention(x, window=None):
    q, k, v = x @ Wq, x @ Wk, x @ Wv
    q, k = rope(q), rope(k)
    mask = j <= i
    if window: mask &= (i – j) < window   # window vs global
    scores = q @ kᵀ / √128
    w = softmax(scores.masked_fill(~mask, –∞))
    out = gate(x) * (w @ v)
    return out @ Wo
```

MoE Layer

```
class MoE:   # one per block; each expert is a small SwiGLU MLP
   Wr = Linear(D, 256)   # the router: one score per expert
   expert = [SwiGLU() for _ in range(256)]   # 256 small MLPs; each token uses 8

   def __call__(self, x):   # x: one token's hidden vector, size D
      p = sigmoid(Wr(x))   # a 0–1 score for each of the 256 experts
      top = topk(p, k=8)   # pick the 8 highest-scoring experts
      w = p[top] / p[top].sum()   # their scores, rescaled to sum to 1
      return Σ w[e] · expert[e](x) for e in top   # weighted average of their outputs, size D
```

```
# Laguna
shared = SwiGLU()   # + one extra expert, no router: every token

return 2.5 · Σ w[e] · expert[e](x) + shared(x)

# balance: a loss over the batch of T tokens
f = one_hot(top).sum(0) / T   # share of tokens sent to e
P = p.mean(0)   # mean router prob for e
loss += α · 256 · (f · P).sum()
```

```
# Beam
bias = zeros(256)   # no gradient
top = topk(p + bias, k=8)   # bias picks, p weighs

# balance: nudge the bias each step; off in mid-training
bias[e] += η · sign(target – load[e])
loss += 5e-4 · 256 · (f · P).sum()   # f, P per sequence
loss += 1e-5 · z_loss(Wr(x))   # small router logits
```

Laguna §3.1 · Beam §2.1.1

Outline

1. Model Factory

Principles

The stack: Titan, Atlas, Blender, Spark, Hive, etc

2. Training details

Architecture

Data: ranking, synthetic, AutoMixer

Infrastructure

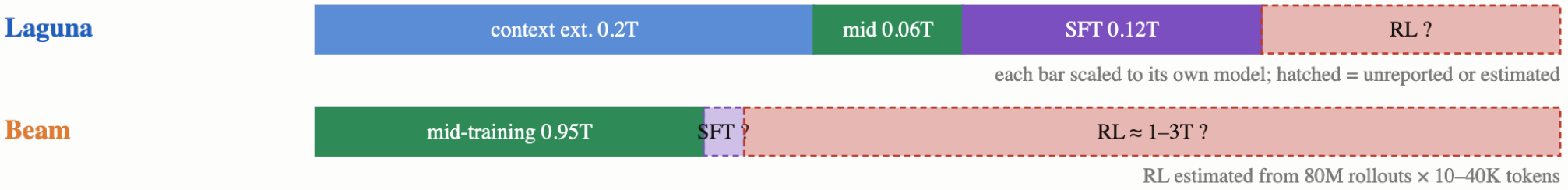
Pre-training

Post-training

Quantization and evaluation

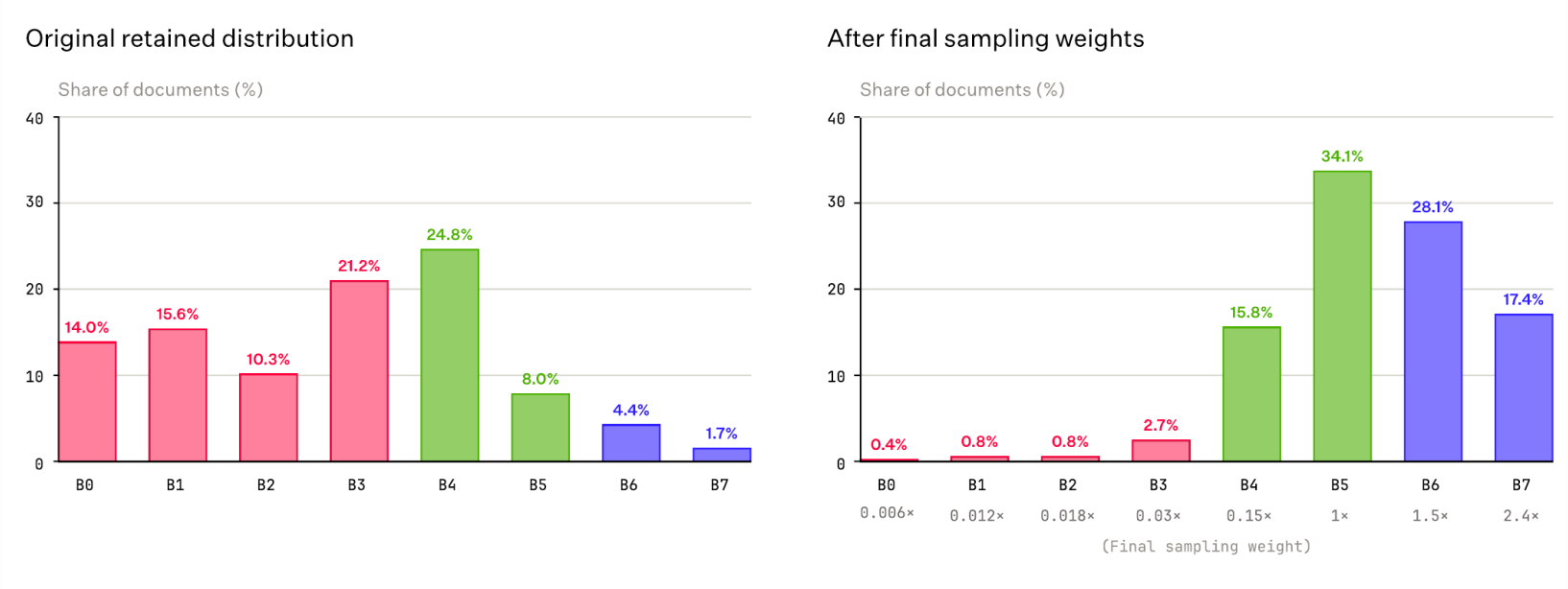
Tokens per Stage

	Laguna XS.2	share	Beam	share
Pre-training	30T+ at 4K	98.7%	23.8T at 16K	96.2%
Chinchilla-optimal	~60B for 3B active → 500× over		~470B for 23B active → 50× over	
Context extension	100B at 32K, 100B at 128K	0.7%	—	—
Mid-training	60B at 128K	0.2%	850B at 32K, 100B at 512K	3.8%
SFT	3 × 40B at 128K	0.4%	Short; not reported	?
RL	Not reported	?	80M+ rollouts	?



Laguna §3.1.1, §4 · Beam §2.1.2, §3, §4.4

Web Data: Weighting by Quality



	Laguna XS.2	Beam
Quality score	Many axes, 8 buckets	One classifier, 20 tiers
Low quality	Kept, weighted down to 0.006×	Bottom 11 tiers dropped
High quality	Weighted up to 2.4×	Top 9 tiers repeated, up to 2×

Laguna §3.2.1, Figure 4 · Beam §2.2

AutoMixer

Finding optimal data mix

```
# researchers
prior = hand mix over 50+ groups
weights = how much each skill matters
# algorithm
mixes = 60 Dirichlet draws centred on prior
scores = train 0.5B on each, score 5 skills
curves = fit mix → score, one per skill
best = mix maximising weights · curves
– distance from prior
check = train 3B on best, 1.5T tokens
```

At 3B / 1.5T vs. hand mix	Change
HumanEval+ (targeted)	+43%
GSM8K (targeted)	+41%
LiveCodeBench (not targeted)	+39%
ARC-C (traded away)	–6.8%

Laguna §3.2.3, Table 3

Beam: Mixing by Scaling Ladder

```
# researchers
mix = hand mix, ablated source by source
change = one proposed edit to the mix
# ladder
ladder = train 300M → 6B, same tokens per param, before and after
gain = compute the baseline would need to match the new loss
accept = gain > 5% on every tier, no eval or stability loss
bundle = fold accepted changes into the next tier → new baseline
```

Final mix: code 35 · English web 35 · STEM 24 · multilingual 5, at 1.73 epochs.

Beam §2.1, §2.2, Table 3

Outline

1. Model Factory

Principles

The stack: Titan, Atlas, Blender, Spark, Hive, etc

2. Training details

Architecture

Data: ranking, synthetic, AutoMixer

Infrastructure

Pre-training

Post-training

Quantization and evaluation

Training Infrastructure

	Laguna	Beam
GPUs	6,144 H200 (M.1) · 2,048 H200 (XS.2)	6,144 GB300
Trainer	Titan: TorchTitan fork, 2,200+ changes	Unicron: Megatron fork
Scheduler	Own k8s scheduler: sub-minute placement, evicts one pod not the gang	Own k8s scheduler: strict priority queue, preempts and migrates
Parallelism	One mesh for the whole model; MoE layers reuse the TP axis for experts	Two meshes: one for attention, one for experts
Token routing	Experts spread over 8 GPUs, so tokens travel to their expert over NVLink, never the network	Same idea, experts spread within one 72-GPU rack

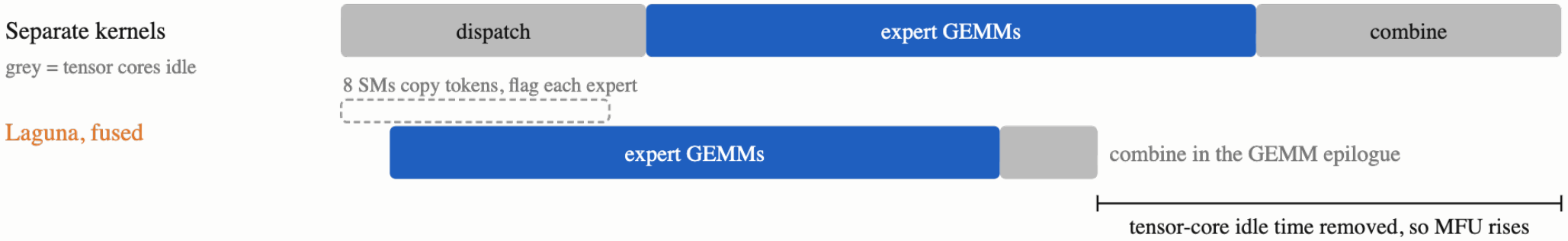
Laguna §2.1.3, §3.1.2 · Beam §6.3, §6.4, §6.6, §6.7

Training Efficiency

Metric	Measured as	Loses time to	Beam
Goodput	time on kept steps ÷ wall-clock	crashes, restarts, rollbacks to the last checkpoint	77% → 92% after fixes
MFU	6 · N_active · tokens/s ÷ peak FLOP/s	comms stalls, pipeline bubbles, small expert GEMMs, memory-bound ops	21% (7.7% at 512K)

Useful share of the cluster ≈ goodput × MFU · Beam: 0.92 × 0.21 ≈ 19% · Laguna reports neither

MFU example: one MoE block on one GPU · widths illustrative, no speedup reported



Beam §6.5, §6.6, Table 26 · Laguna Figure 2, §3.1.2

Outline

1. Model Factory

Principles

The stack: Titan, Atlas, Blender, Spark, Hive, etc

2. Training details

Architecture

Data: ranking, synthetic, AutoMixer

Infrastructure

Pre-training

Post-training

Quantization and evaluation

Recipe

	Laguna XS.2	Beam
Tokens · context	30T+ at 4K	23.8T at 16K
Optimizer	Muon (Moonlight), every stage	AdamC, $\beta_2 = 0.857$
Schedule	WSD: flat, then 1−√ decay over the last 30% to 5% of peak	Cosine to 0.001× peak
Peak LR	5e-4 from a fitted law, lr* ∝ N^−0.46 · D^−0.27	7.6e-4 from a fitted law, η* ∝ N^−α · D^−0.32 · B^0.5
Batch	24M tokens	~50M tokens
Precision	BF16; FP32 norms, RoPE, LM-head gradient reduce	BF16; FP32 residual stream, MoE combine, router
Context extension	100B at 32K + 100B at 128K, YaRN on global layers; 256K free via RoPE scale	None here; 850B at 32K + 100B at 512K in mid-training

Not like for like: Beam is 23.4B active / 501B total, XS.2 is 3B / 33B, so Beam spends ~8× the compute per token and trained on 3× the GPUs. The closer match would be Laguna M.1 (23.4B active), which the paper does not detail.

Laguna §3.1.1 · Beam §2.1, Table 2

Learning Rate Scaling Law

	Laguna	Beam
Grid	4 sizes × 5 token budgets × 6 LRs = 120 runs	4 sizes × one knob α (LR and weight decay together); grid size not reported
Measure	Per (size, budget): loss vs. log LR, fit a parabola, take the minimum	Per model: loss vs. log α, fit a parabola, take the minimum
Fit	Power law in N and D, both exponents from the data	Power law in N, D and B; D and B exponents taken from prior papers, only N fitted
Result	lr* = 10^4.488 · N^–0.46 · D^–0.27, then × √(B/8M) by assumption	η* = C · N^–α · D^–0.32 · B^0.5, and the same for weight decay
Extrapolate to	3B active, 30T tokens → 5.5e-4; ran 5e-4	23B active, 23.8T tokens → 7.6e-4
Reach	Sweep tops out at 2.2B active; 3B is a short hop	Ladder tops out at 6B; 23B is 4× further

N = active parameters, D = total tokens, B = batch in tokens. Neither measures the batch exponent; both borrow the square-root rule.

Laguna §3.1.1, Appendix A.1 · Beam §2.1, Appendix A.3

Outline

1. Model Factory

Principles

The stack: Titan, Atlas, Blender, Spark, Hive, etc

2. Training details

Architecture

Data: ranking, synthetic, AutoMixer

Infrastructure

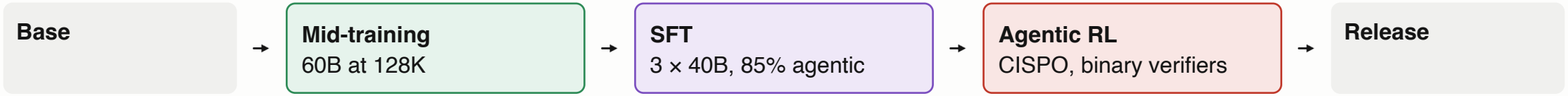
Pre-training

Post-training

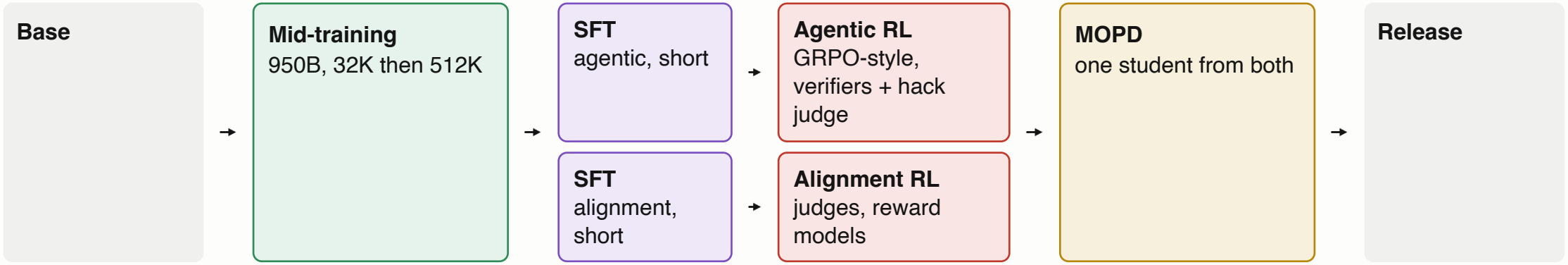
Quantization and evaluation

Post-training Pipelines

Laguna XS.2



Beam



Mid-training

	Laguna XS.2	Beam
Tokens · context	60B at 128K	850B at 32K, then 100B at 512K
Purpose	Template, tools, thinking	Knowledge, long context
Mix	40% reasoning, 30% agentic, 30% chat	45% coding, 30% STEM, 15% general, 10% recall
Schedule	Small cosine, optimizer kept	Restart per stage, peak at 5% of pre-training
Router	Balancing kept on: aux loss stays	Balancing switched off: experts left to specialise

Laguna §4.2 · Beam §3

SFT

	Laguna XS.2	Beam
Size	3 × 40B tokens	“Short”, not reported
Purpose	Capability	Format and reasoning
Data	Teacher rollouts on tasks mined from 236k commits	An earlier Beam’s rollouts on the RL task pool, filtered
Mix	~85% agentic coding, the rest non-agentic	All RL domains
Extras	Instruction-following requirements appended and judged	Separate SFT for the alignment expert

Laguna §4.3 · Beam §4.1.1, §4.2.2

Thinking Control

	Laguna XS.2	Beam
Knob	Thinking on or off	Effort level, 32 to 512
In the prompt as	An open or a closed think tag	An integer; 512 ≈ 40K output tokens in practice
Trained by	Mid-training data, balanced across tools	RL: over budget gets no reward, shorter wins ties

Laguna §4.1, §4.2 · Beam §4.1.2

RL Algorithm

	Laguna XS.2	Beam
Family	CISPO	GRPO-style
Baseline	Leave-one-out mean, length-weighted	Group mean
Clip	Importance ratio to [0, 5]	Not reported
MoE detail	—	Routing replay: trainer reuses inference's expert choices

Laguna §4.4.1 · Beam §4.1.2

Reward

	Laguna XS.2	Beam
Correctness	Binary verifier	Binary verifier, then a hack judge on passes
Format	–0.1 malformed tool call, last turn only	—
Behaviour	–0.1 too few tool calls; –0.05 per token on tool errors	Shorter wins among correct answers
Budget	Timeout → 0	Over effort budget → 0
Learned reward	None	Judges and reward models for the alignment expert
Chain of thought	—	Monitored, not optimised

BEAM'S JUDGES

Hack judge	Passing agentic rollouts, against a hacking rubric
Calibration judge	Final answer vs. verified gold; abstaining scores half
Rubric judge	~22 instruction-following rubrics per prompt, all must pass
Generative RM	Open-ended chat quality; recalibrated on human labels every 2 weeks

Laguna §4.4.2 · Beam §4.1.2, §4.2.3

Outline

1. Model Factory

Principles

The stack: Titan, Atlas, Blender, Spark, Hive, etc

2. Training details

Architecture

Data: ranking, synthetic, AutoMixer

Infrastructure

Pre-training

Post-training

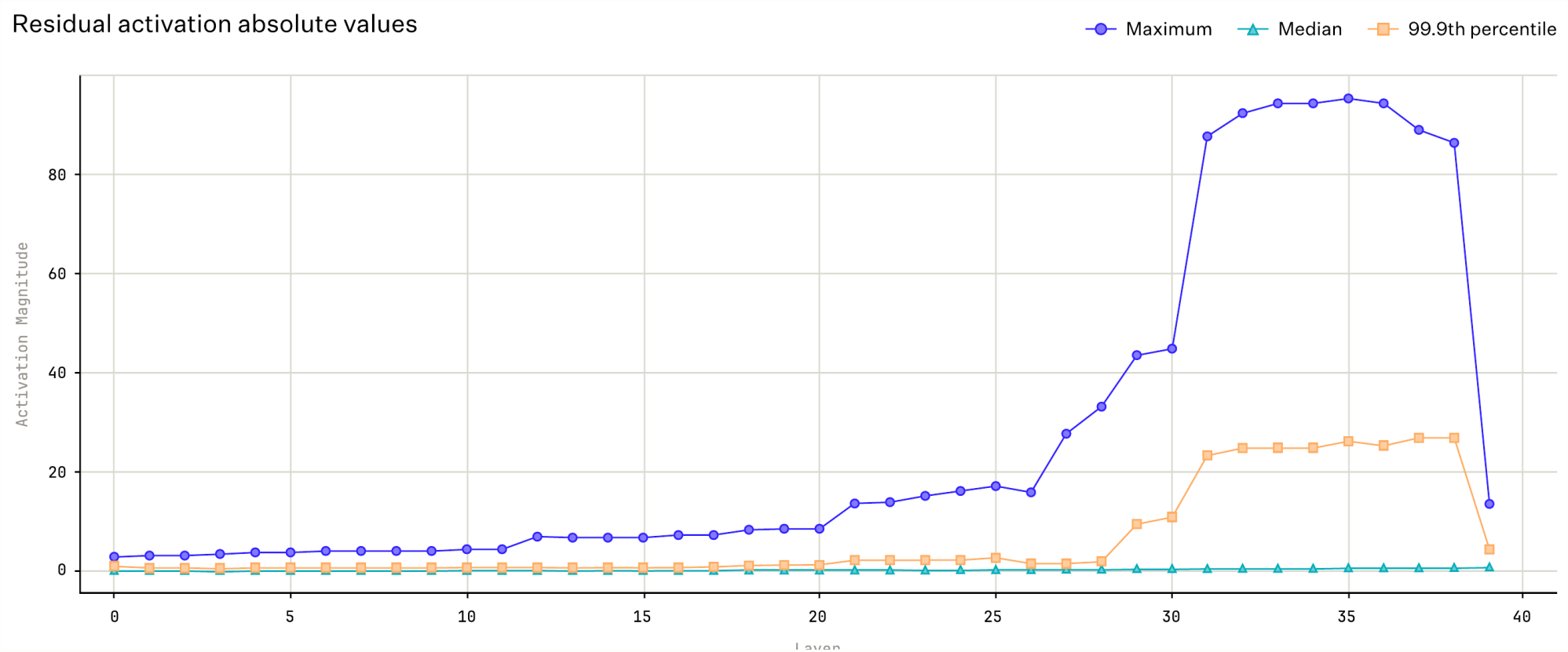
Quantization and evaluation

Quantization

	Laguna XS.2	Beam
Formats	FP8, INT4, NVFP4	FP8, NVFP4
FP8	Rotation and block scales	Round-to-nearest, no calibration
4-bit	INT4 everywhere but layers 31–40; NVFP4 by distillation	NVFP4 on experts only, Hessian-calibrated
Calibration data	128 agentic trajectories	1,000 agentic trajectories, 32.8M tokens
Picked by	Agentic benchmarks	KL to BF16 on held-out agentic sequences, then benchmarks
Recovery	“No measurable loss” at FP8; number not given	99.9% FP8, 99.0% NVFP4
Where it broke	Agentic evals first, single-turn barely moved	Terminal-Bench and SWE-bench Verified

Laguna §5 · Beam §4.5

Why INT4 Stops at Layer 30



Residual-stream outliers accumulate from ~layer 30 of 40. Layers 1–30 go to INT4, 31–40 stay INT8.

Laguna Figure 7, §5

Evaluation Protocol

	Laguna XS.2	Beam
Sampling	T 1.0, top-k 20, thinking on	T 0.7, top-p 0.99, effort 512
Runs	Mean of 4	Not reported
Context	256K	1M
Hack audit	Judge plus manual review on every reported run	Integrity judge during RL

Laguna §6.2 · Beam §5.1, §2.2.4, §4.1.5

Discussion